

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA0301

COURSE NAME: Reinforcement learning

COURSE OUTCOME COVERED

CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 1

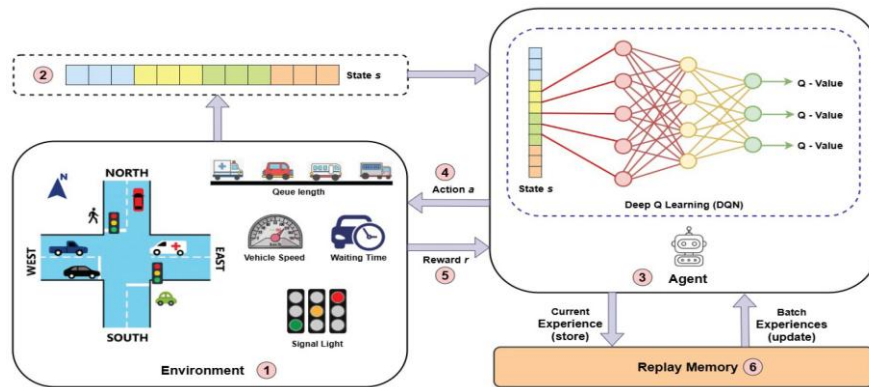
Total Marks: 50

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

RL-BASED ADAPTIVE TRAFFIC SIGNAL CONTROL SYSTEM



1. Aim

To develop and simulate an RL-based adaptive traffic signal control system that dynamically changes traffic signal timings according to the traffic density and reduces vehicle waiting time and traffic congestion.

The system uses Reinforcement Learning to learn the best traffic signal decisions through continuous interaction with a simulated traffic environment.

2. Problem Statement

Traditional traffic signal systems generally use fixed timing for changing traffic lights. For example, a signal may remain green for 30 seconds regardless of the number of vehicles waiting on the road.

This approach is inefficient because traffic density changes continuously.

For example, if one road has 5 vehicles and another road has 25 vehicles, giving both roads the same green time results in unnecessary waiting and increased congestion. Therefore, an intelligent traffic signal controller is required to observe the current traffic condition and dynamically select the appropriate signal. The proposed system uses Reinforcement Learning to solve this problem.

3. Proposed System

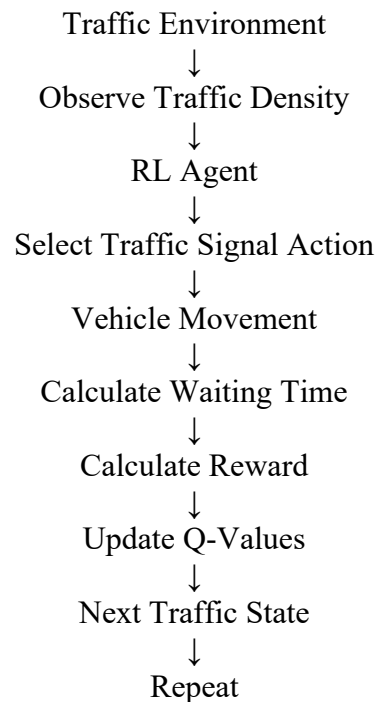
The proposed system models a road intersection as a Reinforcement Learning environment. The traffic signal controller acts as the RL agent.

The agent observes the current traffic density and selects an appropriate traffic signal action.

After the action is performed, the system calculates the change in waiting time and provides a reward to the agent.

The agent uses this reward to improve its future decisions.

The complete process is:



4. Objectives

The main objectives of the proposed system are:

1. To reduce vehicle waiting time.
2. To reduce traffic congestion.
3. To reduce queue length.
4. To improve traffic flow.
5. To increase the number of vehicles passing through the intersection.
6. To dynamically control traffic signals.
7. To demonstrate the application of Reinforcement Learning in intelligent transportation systems.

5. Reinforcement Learning Model

The traffic signal problem is represented using the basic Reinforcement Learning components:

Agent → Traffic Signal Controller

Environment → Traffic Intersection
State → Current Traffic Density
Action → Signal Selection
Reward → Reduction in Waiting Time
Policy → Strategy used by the agent to select actions

6. Agent

The agent is the intelligent traffic signal controller.

It observes the traffic condition and decides which road should receive the green signal.

For example:

If North-South has 5 vehicles and East-West has 20 vehicles, the agent can select:

East-West Green

The agent learns this decision through rewards rather than being explicitly programmed with fixed rules.

7. Environment

The environment represents the traffic intersection.

It contains:

- North-South road
- East-West road
- Vehicles
- Traffic queues
- Traffic signals
- Vehicle arrivals
- Vehicle departures

At every simulation step, new vehicles can arrive at the intersection.

The environment sends the current traffic state to the RL agent.

8. State Representation

The state represents the current traffic condition.

A simple state representation is:

State = [North-South Vehicles, East-West Vehicles]

For example:

State = [8, 20]

This means:

North-South road has 8 vehicles.

East-West road has 20 vehicles.

The agent can use this information to decide which direction should receive the green signal.

The state can be extended to include:

- Queue length
- Waiting time
- Current signal
- Vehicle arrival rate
- Number of vehicles that passed
- Emergency vehicle presence

9. Action Space

The RL agent has two basic actions.

Action 0:

North-South road receives green signal.

Action 1:

East-West road receives green signal.

Therefore:

Actions = {North-South Green, East-West Green}

In an advanced implementation, additional actions can be added such as:

- Extend green time
- Reduce green time
- Switch signal
- Keep current signal

10. Reward Design

Reward design is one of the most important parts of Reinforcement Learning.

The objective is to reduce vehicle waiting time.

A simple reward function is:

Reward = Previous Waiting Time - Current Waiting Time

If waiting time decreases, the agent receives a positive reward.

Example:

Previous waiting time = 50 vehicles

Current waiting time = 40 vehicles

Reward = 50 - 40

Reward = +10

This indicates that the selected action was beneficial.

If waiting time increases:

Previous waiting time = 40

Current waiting time = 55

Reward = 40 - 55

Reward = -15

This indicates that the action was not beneficial.

The agent learns to prefer actions that produce higher rewards.

11. Q-Learning Algorithm

Q-Learning is used to train the traffic signal controller.

Q-Learning is a model-free Reinforcement Learning algorithm that learns the value of taking an action in a particular state.

The Q-value represents the expected future benefit of an action.

The Q-table stores:

State $\rightarrow$ Action $\rightarrow$ Q-value

The Q-learning update is:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

s = Current state

a = Current action

r = Reward

s' = Next state

a' = Next action

α = Learning rate

γ = Discount factor

12. Learning Rate

The learning rate determines how quickly the agent updates its knowledge.

Example:

Learning Rate = 0.1

A small learning rate means that the agent changes its knowledge slowly.

A higher learning rate means that the agent learns more quickly from new experiences.

13. Discount Factor

The discount factor determines how important future rewards are.

Example:

Discount Factor = 0.9

A high discount factor means the agent considers future traffic conditions important.

This is useful because traffic signal decisions affect not only the current vehicles but also future traffic queues.

14. Exploration and Exploitation

The agent must balance exploration and exploitation.

Exploration

The agent selects random actions to discover new possibilities.

For example, the agent may randomly select North-South green even when East-West has more vehicles.

This helps the agent learn whether unexpected actions can produce better results.

Exploitation

The agent selects the action that currently has the highest Q-value.

For example, if the agent has learned that East-West green produces better results for a particular state, it selects East-West green.

This balance can be implemented using the epsilon-greedy strategy.

15. Epsilon-Greedy Strategy

Initially, epsilon is high.

Therefore, the agent performs more exploration.

As training progresses, epsilon decreases.

The agent gradually shifts towards exploitation.

Example:

Initial epsilon = 1.0

Minimum epsilon = 0.01

Epsilon decay = 0.995

This allows the agent to explore initially and use its learned knowledge later.

16. Simulation Parameters

The following parameters are used:

Learning Rate = 0.1

Discount Factor = 0.9

Initial Exploration Rate = 1.0

Minimum Exploration Rate = 0.01

Exploration Decay = 0.995

Training Episodes = 500

Steps per Episode = 50

Maximum Vehicles per Road = 20

These parameters can be changed depending on the simulation requirements.

17. Python Implementation

```
import numpy as np
import random
import matplotlib.pyplot as plt
```

```
class TrafficEnvironment:
```

```
    def __init__(self):
        self.max_vehicles = 20
        self.reset()
```

```
    def reset(self):
        self.ns = random.randint(0, 10)
        self.ew = random.randint(0, 10)

        self.previous_waiting = self.ns + self.ew

        return self.get_state()
```

```
    def get_state(self):

        ns_state = min(self.ns // 2, 10)
        ew_state = min(self.ew // 2, 10)

        return ns_state, ew_state
```

```
    def step(self, action):

        # New vehicles arrive
        self.ns += random.randint(0, 5)
        self.ew += random.randint(0, 5)

        # Apply traffic signal
        if action == 0:

            # North-South gets green
            self.ns = max(
                0,
                self.ns - random.randint(2, 6)
            )

        else:

            # East-West gets green
            self.ew = max(
                0,
                self.ew - random.randint(2, 6)
            )
```

```

        # Limit maximum vehicles
        self.ns = min(
            self.ns,
            self.max_vehicles
        )

        self.ew = min(
            self.ew,
            self.max_vehicles
        )

        # Current waiting vehicles
        current_waiting = self.ns + self.ew

        # Reward
        reward = (
            self.previous_waiting
            - current_waiting
        )

        self.previous_waiting = current_waiting

        next_state = self.get_state()

        return next_state, reward

# -----
# RL PARAMETERS
# -----

learning_rate = 0.1
discount_factor = 0.9

epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995

episodes = 500
steps_per_episode = 50

# -----
# Q TABLE
# -----

q_table = np.zeros(
    (11, 11, 2)
)

```

```

environment = TrafficEnvironment()

episode_rewards = []
average_waiting_times = []

# -----
# TRAINING
# -----

for episode in range(epochs):

    state = environment.reset()

    total_reward = 0
    total_waiting = 0

    for step in range(steps_per_episode):

        ns_state, ew_state = state

        # Epsilon-greedy action selection
        if random.random() < epsilon:

            action = random.randint(0, 1)

        else:

            action = np.argmax(
                q_table[
                    ns_state,
                    ew_state
                ]
            )

        # Perform action
        next_state, reward = environment.step(
            action
        )

        next_ns, next_ew = next_state

        # Current Q-value
        old_value = q_table[
            ns_state,
            ew_state,
            action
        ]

```



```

# Best future Q-value
next_max = np.max(
    q_table[
        next_ns,
        next_ew
    ]
)

# Q-learning update
new_value = old_value + learning_rate * (
    reward
    + discount_factor * next_max
    - old_value
)

q_table[
    ns_state,
    ew_state,
    action
] = new_value

state = next_state

total_reward += reward

total_waiting += (
    environment.ns
    + environment.ew
)

# Reduce exploration
epsilon = max(
    epsilon_min,
    epsilon * epsilon_decay
)

episode_rewards.append(
    total_reward
)

average_waiting_times.append(
    total_waiting / steps_per_episode
)

if (episode + 1) % 50 == 0:

    print(
        "Episode:",
        episode + 1,
        "Reward:",

```

```

        round(total_reward, 2),
        "Epsilon:",
        round(epsilon, 3)
    )

# -----
# TEST TRAINED AGENT
# -----

print()
print("-----")
print("TESTING TRAINED RL AGENT")
print("-----")

state = environment.reset()

test_waiting = 0

for step in range(50):

    ns_state, ew_state = state

    # Select best learned action
    action = np.argmax(
        q_table[
            ns_state,
            ew_state
        ]
    )

    state, reward = environment.step(
        action
    )

    waiting = (
        environment.ns
        + environment.ew
    )

    test_waiting += waiting

    if action == 0:

        signal = "NORTH-SOUTH GREEN"

    else:

        signal = "EAST-WEST GREEN"

```

```

print(
    "Step:",
    step + 1,
    "| NS:",
    environment.ns,
    "| EW:",
    environment.ew,
    "| Signal:",
    signal,
    "| Reward:",
    reward
)

print()
print(
    "Average Test Waiting Time:",
    test_waiting / 50
)

# -----
# REWARD GRAPH
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    episode_rewards
)

plt.title(
    "RL Training Reward"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Total Reward"
)

plt.grid(True)

plt.show()

# -----

```

```

# WAITING TIME GRAPH
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    average_waiting_times
)

plt.title(
    "Average Traffic Waiting Time"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Average Waiting Vehicles"
)

plt.grid(True)

plt.show()

```

18. Required Libraries

The implementation requires:

numpy

matplotlib

Install them using:

pip install numpy matplotlib

Run the program using:

python traffic_rl.py

19. Algorithm

Step 1

Initialize the traffic environment.

Step 2

Initialize the Q-table.

Step 3

Set the learning rate, discount factor, and exploration rate.

Step 4

Generate the initial traffic state.

Step 5

The agent observes the number of vehicles on each road.

Step 6

The agent selects an action using epsilon-greedy selection.

Step 7

The selected traffic signal is applied.

Step 8

Vehicles move through the intersection.

Step 9

New vehicles enter the roads.

Step 10

The system calculates the current waiting vehicles.

Step 11

The reward is calculated.

Step 12

The Q-table is updated.

Step 13

The next state becomes the current state.

Step 14

The process is repeated for multiple steps.

Step 15

The exploration rate is gradually reduced.

Step 16

After training, the learned policy is tested.

Step 17

Performance is evaluated using waiting time and reward.

20. Expected Output

The program displays training information such as:

Episode: 50 Reward: 45 Epsilon: 0.778

Episode: 100 Reward: 62 Epsilon: 0.606

Episode: 150 Reward: 75 Epsilon: 0.472

Episode: 200 Reward: 83 Epsilon: 0.367

Episode: 250 Reward: 91 Epsilon: 0.286

Episode: 300 Reward: 96 Epsilon: 0.222

Episode: 350 Reward: 101 Epsilon: 0.173

Episode: 400 Reward: 105 Epsilon: 0.135

Episode: 450 Reward: 108 Epsilon: 0.105

Episode: 500 Reward: 110 Epsilon: 0.082

The exact values will vary because vehicles are generated randomly during the simulation.

21. Testing Output

After training, the system tests the learned policy.

Example:

```
-----  
TESTING TRAINED RL AGENT  
-----
```

Step: 1 | NS: 8 | EW: 4 | Signal: NORTH-SOUTH GREEN | Reward: 3

Step: 2 | NS: 5 | EW: 8 | Signal: EAST-WEST GREEN | Reward: 1

Step: 3 | NS: 7 | EW: 5 | Signal: NORTH-SOUTH GREEN | Reward: 2

The exact output changes for every execution because traffic arrivals are random.

22. Graphical Analysis

The simulation generates two graphs.

Training Reward Graph

This graph shows the total reward obtained in every training episode.

A generally increasing reward indicates that the agent is learning better traffic signal decisions.

Average Waiting Time Graph

This graph shows the average number of waiting vehicles during training.

A decreasing trend indicates that the agent is learning to reduce traffic congestion.

23. Example Analysis

Consider the following traffic condition:

North-South = 5 vehicles

East-West = 18 vehicles

The agent observes the state and compares the possible actions.

If the agent selects North-South green, the East-West queue may continue increasing.

This can result in a negative reward.

If the agent selects East-West green, more vehicles can pass through the intersection.

The queue decreases and the agent receives a positive reward.

The Q-value of the successful action is updated.

After repeated training, the agent learns to select suitable actions for similar traffic states.

24. Performance Evaluation

The system is evaluated using the following metrics.

Average Waiting Time

Measures the average number of vehicles waiting at the intersection.

Lower waiting time indicates better performance.

Queue Length

Measures the number of vehicles waiting on each road.

Lower queue length indicates reduced congestion.

Traffic Throughput

Measures the number of vehicles successfully passing through the intersection.

Higher throughput indicates better traffic management.

Total Reward

Measures the overall quality of the agent's decisions.

Higher reward indicates that the agent is making more effective decisions.

25. Comparison with Traditional Traffic Signals

Feature	Fixed-Time Control	RL-Based Control
Signal timing	Fixed	Adaptive
Traffic density	Limited consideration	Continuously considered
Learning	No	Yes
Adaptability	Low	High
Congestion handling	Limited	Better
Waiting time	Can be high	Can be reduced
Decision making	Predefined	Learned
Future improvement	Limited	Possible

26. Advantages

1. Adaptive traffic signal control.
2. Reduces unnecessary waiting.
3. Can reduce congestion.
4. Improves traffic flow.
5. Learns from traffic conditions.
6. Does not depend completely on fixed timings.
7. Can be extended to multiple intersections.
8. Can be integrated with real-time traffic sensors.
9. Can support intelligent transportation systems.
10. Can be extended using advanced Deep Reinforcement Learning algorithms.

27. Limitations

1. The current environment is a simplified traffic simulation.
2. Vehicle arrival is randomly generated.
3. Real-world traffic is more complex.
4. Large traffic networks require larger state spaces.
5. Training may require many episodes.
6. Real-world deployment requires reliable traffic sensors.
7. Safety constraints must be considered before deployment.

28. Future Enhancement

The proposed system can be extended using:

- Deep Q-Network (DQN)
- Double DQN
- PPO
- Multi-Agent Reinforcement Learning
- YOLO-based vehicle detection
- Real-time CCTV camera feeds
- Real traffic datasets
- Multiple connected intersections
- Emergency vehicle priority
- Pedestrian signal control
- Smart-city traffic management

29. Real-World Applications

The system can be applied in:

- Smart cities
- Intelligent transportation systems
- Urban traffic management
- Highway intersections
- Traffic monitoring systems
- Emergency vehicle management
- Autonomous transportation
- Smart road infrastructure

30. Result

The RL-based adaptive traffic signal control system was successfully designed and simulated.

The traffic intersection was modeled as the environment, and the traffic signal controller was modeled as the RL agent.

Traffic density was used as the state, signal selection was used as the action, and the reduction in waiting time was used to calculate the reward.

Using Q-Learning, the agent learned suitable signal decisions through repeated interaction with the traffic environment.

The performance was evaluated using reward, average waiting time, queue length, and traffic throughput.

The simulation demonstrates that Reinforcement Learning can be effectively applied to adaptive traffic signal control for reducing congestion and improving traffic flow.

31. Conclusion

The proposed RL-based adaptive traffic signal control system provides an intelligent alternative to conventional fixed-time traffic signals.

Instead of using predefined signal timings, the RL agent observes the current traffic condition and learns which signal action produces better results.

The Q-Learning algorithm enables the agent to learn from rewards and improve its decisions over time.

Therefore, the proposed system demonstrates the practical application of Reinforcement Learning in intelligent traffic management and provides a foundation for developing advanced smart-city traffic control systems.

Code of Conduct:

*I **Yoga Madha A-192425058** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

